

Car Inspector

Projet BDR – Rendu final

Axel Pittet, Adam Gruber, Vernet Bénédicte
26/01/2025

Table des matières

1. Introduction	3
2. Modèle EA	3
2.1 Choix de conception.....	3
2.2 Contraintes d'intégrité	5
2.2.1 Contrainte de nullité.....	5
2.2.2 Contraintes de date	5
2.2.3 Contraintes numériques	5
2.3 Schéma EA	6
3. Modèle relationnel	6
4. Scripts SQL	8
4.1 Contraintes CHECK	8
Car.....	8
Car_Option	9
Modification	9
Performance.....	9
Engine	9
Gas.....	10
Electric.....	10
Transmission	10
Brake	10
Review.....	11
4.2 Types de données.....	11
Brand.....	11
Image	11
AppUser	11
Engine	11
Transmission	12
5. Description de l'application réalisée	12
5.1 Authentification.....	12
5.2 Recherche de voitures	13
5.3 Fiche détaillée d'une voiture	13
5.4 Gestion des reviews.....	14
5.5 Proposition de modifications	14

5.6 Gestion des utilisateurs·ices pour les administrateurs·ices.....	15
6. Améliorations futures	16
7. Conclusion.....	17
8. Annexes.....	17
8.1 Guide d'installation.....	17

1. Introduction

Ce projet a pour ambition de concevoir un dictionnaire automobile interactif, offrant une centralisation des informations essentielles sur les performances et les fonctionnalités principales des voitures. Ce dictionnaire permettra à un·e utilisateur·ice de rechercher un modèle précis de voiture et d'accéder à une vue d'ensemble détaillée de ses caractéristiques.

Pour cela, nous avons décidé de réaliser un site web utilisant les technologies suivantes : Django Rest Framework pour le backend, VueJS (JavaScript) pour le frontend et PostgreSQL pour notre base de données.

Le but était qu'un·e utilisateur·ice puisse accéder à notre site web afin de se renseigner avant d'acheter la voiture la plus adaptée à ses besoins. Nous voulions qu'il soit possible de rechercher une voiture par son nom, sa marque, son type de moteur ou encore sa puissance et que notre site puisse lui retourner la liste des voitures correspondant à sa recherche. L'utilisateur·ice pourrait ensuite cliquer sur une voiture afin d'obtenir plus d'informations la concernant, comme, par exemple son prix, le type de son moteur ou encore sa date de sortie. Iel pourrait ensuite laisser une review avec une note et un commentaire sur une spécification particulière et consulter celles laissées par d'autres utilisateur·ice. Iel pourrait également suggérer une modification de la voiture, un·e utilisateur·ice admin pourrait ensuite l'accepter, ou non.

Afin de réaliser ces objectifs, nous avons suivi un procédé en étapes bien définies ; en commençant par réaliser un modèle EA et un modèle relationnel. Nous avons ensuite créé des scripts DDL et DML afin de créer les tables correspondantes et de les remplir avec des requêtes SQL. Pour finir, nous avons réalisé le site web en implémentant un backend et un frontend.

2. Modèle EA

2.1 Choix de conception

Notre table centrale est la table *Car*, celle-ci répertorie le modèle de la voiture, le nombre de places et de portes, la date de sortie, le prix de base de la voiture (sans options, ni spécifications particulières) et une à plusieurs images de la voiture en question.

Cette table est reliée à la table *Option* afin de regrouper toutes les possibilités de modèles de voitures en fonction des différentes options disponibles. La table *Option* représente par exemple l'option sièges chauffants, la couleur de la voiture ou encore la possibilité d'avoir un écran tactile. Le prix de la voiture va donc varier en fonction du nombre et de quelles options ont été choisies. C'est pour cela que nous avons ajouté un attribut d'association *OptionPrice* puisque celui-ci diffère en fonction de l'option et du modèle de la voiture.

Ensuite, nous avons créé une table *Specification* liée aux tables *Car*, *Brake*, *Engine*, *Transmission* et *Performance*. Cette table regroupera les différentes clés étrangères

de ces derniers afin de stocker toutes les possibilités existantes. Certes, cette table est optionnelle puisqu'elle ne contient pas d'attributs particuliers, mais elle nous permet de mieux visualiser et organiser notre schéma. Ainsi, nous pouvons directement accéder depuis les spécifications à tous les types de freins, moteurs et transmissions par modèle de voiture. Selon la configuration des spécifications, ses performances seront ensuite stockées dans la table *Performance*.

Nous utiliserons également la table *Specification* afin de calculer le prix total à afficher en reprenant les prix (grâce à l'attribut *price*) de chacun de ses composants (*Brake*, *Engine* et *Transmission*), des options choisies (*Option*) et de l'attribut *defaultPrice* de *Car*.

Une voiture peut être de trois types différents : électrique, à essence ou hybride. Afin de représenter cela, nous avons décidé de créer un héritage sur *Engine*. Les tables *Gas* et *Electric* nous permettent d'apporter des informations spécifiques à ses moteurs.

La relation entre *Specification* et *Engine* démontre que nous pouvons avoir plusieurs moteurs sur une seule voiture. Cela est dû aux véhicules hybrides et électriques qui en possèdent plusieurs. Par exemple, la Tesla Model S Plaid en contient 3.

Une voiture peut sortir avec un nouveau modèle, et ce modèle peut avoir plusieurs versions. Par exemple, avec des moteurs de puissances différentes. Pour représenter cela, nous avons mis une cardinalité de 1...* vers les spécifications.

Nous avons décidé de stocker sur la table *Transmission* le type de la voiture, soit manuelle ou automatique dans l'attribut *type*. En ce qui concerne l'attribut *drivetrain*, il peut prendre les valeurs suivantes correspondent aux différentes configurations de transmission :

- RWD : Rear Wheel Drive – Propulsion arrière
- FWD : Front Wheel Drive – Traction avant
- AWD : All Wheel Drive – Transmission intégrale 4 roues motrices
- 4WD : Four Wheel Drive – Transmission intégrale 4x4

Une *Modification* ne modifie pas directement une voiture ou un composant, il s'agit uniquement d'un texte écrit par un utilisateur afin de décrire l'erreur qu'il pense avoir observée et elle sera ensuite passée en revue par un *superUser* qui remplira le champ *isAccepted* selon son expertise. Dans le cas positif, cela sera alors le travail de l'admin, *superUser*, d'aller manuellement modifier la base de données afin de la mettre à jour.

En ce qui concerne les unités, les attributs de prix seront tous mesurés en CHF. Les attributs *isAccepted* (*Modification*), *isSuperUser* (*User*) et *abs* (*Brake*) sont des booléens. Les mesures de distance seront effectuées en km. L'attribut *zeroToHundred* dans la table *Performance* est la vitesse à laquelle la voiture atteint la vitesse de 100km/h et celui-ci sera stocké en secondes. Dans la même table, l'attribut *maxSpeed* va être stocké en km/h. L'attribut *engineDisplacement* va être stocké dans la table *Gas*, va être stocké en Litres.

2.2 Contraintes d'intégrité

2.2.1 Contrainte de nullité

La majorité de nos tables contiennent des attributs qui ne peuvent pas être NULL. Nous allons donc préciser tous ce qui peuvent prendre la valeur NULL. Un scénario plausible qui explique ce choix est ajouté à chaque contrainte.

1. *Review* : L'attribut *content* peut prendre la valeur de NULL.

Un·e utilisateur·ice pourrait écrire une review d'une voiture en mettant toutes les informations nécessaires dans le titre.

2. *Brand* : L'attribut *image* peut prendre la valeur de NULL.

Lorsqu'un·e utilisateur·ice recherche une voiture, il va pouvoir voir quelle marque l'a produite mais il n'est pas forcément essentiel d'avoir une image représentant le logo de la marque.

3. *Modification* : L'attribut booléen *isAccepted* peut prendre la valeur de NULL.

Lors de la création d'une modification, cet attribut sera laissé vide afin de pouvoir classer une modification en 3 types distincts selon la valeur de *isAccepted* :

- (NULL) : Les modifications pas encore passées en revue.
- *True* : Les modifications acceptées.
- *False* : Les modifications refusées.

2.2.2 Contraintes de date

1. La date d'une *Review* ne peut pas être plus ancienne que la *releaseDate* de la voiture à laquelle elle est liée. La date ne pourra pas non plus être au-delà d'aujourd'hui
2. La date de sortie d'une voiture (*releaseDate*) ne peut non plus être postérieure à la date d'aujourd'hui

2.2.3 Contraintes numériques

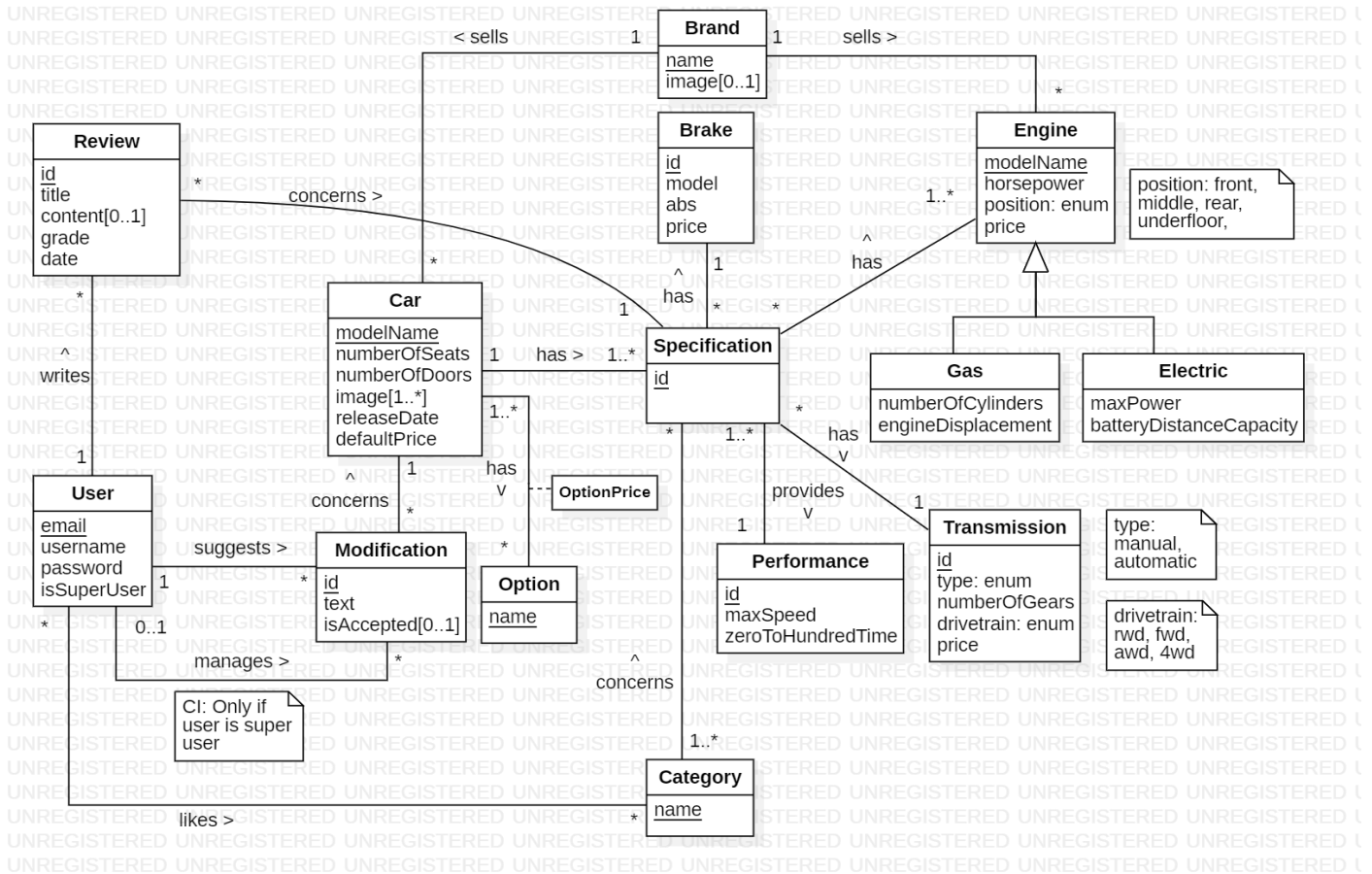
Dans le schéma UML, tous les attributs qui contiennent des nombres doivent être positifs ou nuls. Aucun de ces attributs ne pourra contenir de valeurs négatives, car cela n'aurait pas de sens dans le contexte des données qu'ils représentent. Cette contrainte garantit la cohérence et la validité des informations stockées dans la base de données.

L'attribut *grade*, permet de donner la note de la *Review* écrite par l'utilisateur·ice. La note peut varier entre 1 et 5, représentant le système de la notation par étoiles. Cette valeur doit donc être entière, elle ne peut pas contenir de virgule.

Il en va de même pour les attributs *numberOfSeats* (*Car*), *numberOfDoors* (*Car*), *numberOfCylinders* (*Gas*) et *numberOfGears* (*Transmission*). Tous ses attributs sont de type entier.

2.3 Schéma EA

Pour plus de lisibilité, vous trouverez également notre schéma EA en annexe.



3. Modèle relationnel

Nous avons ensuite réalisé un modèle relationnel basé sur notre schéma UML.

Brand(name, image)

Car(modelName, numberOfSeats, numberOfDoors, releaseDate, defaultPrice, nameBrand)

Car.nameBrand references Brand.name

Option(name)

Car_Option(modelNameCar, nameOption, optionPrice)

Car_Option.modelNameCar references Car.modelName

Car_Option.nameOption references Option.name

Image(modelNameCar, image)

Image.modelNameCar references Car.modelName

User(email, username, password, isSuperUser)

Modification(id, text, isAccepted, modelNameCar, emailUserSuggests,
emailUserManages)

Modification.modelNameCar references Car.modelName,

Modification.modelNameCar NOT NULL

Modification.emailUserSuggests references User.email,

Modification.emailUserSuggests NOT NULL

Modification.emailUserManages references User.email,

Modification.emailUserManages NOT NULL

Category(name)

User_Category(emailUser, nameCategory)

User_Category.emailUser references User.email

User_Category.nameCategory references Category.name

Performance(id, maxSpeed, zeroToHundredTime)

Engine(modelName, horsepower, position, price, nameBrand)

Engine.nameBrand references Brand.name, Engine.nameBrand NOT NULL

Gas(modelNameEngine, numberOfCylinders, engineDisplacement)

Gas.modelNameEngine references Engine.modelName

Electric(modelNameEngine, maxPower, batteryDistanceCapacity)

Electric.modelNameEngine references Engine.modelName

Transmission(id, type, numberOfGears, drivetrain, price)

Brake(id, model, abs, price)

Specification(id, modelNameCar, idBrake, idTransmission, idPerformance)
Specification.modelNameCar references Car.modelName,
Specification.modelNameCar NOT NULL
Specification.idBrake references Brake.id, Specification.idBrake NOT NULL
Specification.idTransmission references Transmission.id,
Specification.idTransmission NOT NULL
Specification.idPerformance references Performance.id, Specification.idPerformance NOT NULL

Specification_Engine(idSpecification, modelNameEngine)
Specification_Engine.idSpecification references Specification.id
Specification_Engine.modelNameEngine references Engine.modelName,
Specification_Engine.modelNameEngine NOT NULL

Review(id, title, content, grade, date, idSpecification, emailUser)
Review.idSpecification references Specification.id, Review.idSpecification NOT NULL
Review.emailUser references User.email, Review.emailUser NOT NULL

Category_Specification(nameCategory, idSpecification)
Category_Specification.nameCategory references Category.name, Category.name NOT NULL
Category_Specification.idSpecification references Specification.id

4. Scripts SQL

Nos scripts SQL complets se trouvent dans l'annexe de notre projet. Nous en avons réalisé 2 : un DDL afin de créer les tables et un DML afin de les remplir avec des tuples initiaux.

Le nom de la table *User* a été changé en *AppUser* pour la réalisation de nos scripts car le mot *User* est un mot réservé en SQL.

4.1 Contraintes CHECK

Lors de la création des tables, nous avons dû rédiger plusieurs contraintes afin que l'on ne puisse pas ajouter des valeurs incohérentes dans nos tables.

Car

1. CK_Car_numberOfSeats CHECK (numberOfSeats >= 1)

Cette contrainte garantit qu'une voiture a au moins un siège. Une voiture ne peut pas être définie avec zéro ou un nombre négatif de sièges.

2. CK_Car_releaseDate CHECK (releaseDate <= CURRENT_DATE)

Cette contrainte s'assure que la date de sortie d'une voiture ne peut pas être dans le futur. Cela reflète la réalité où une voiture ne peut être ajoutée qu'après sa sortie officielle.

3. CK_Car_defaultPrice CHECK (defaultPrice > 0)

Cette contrainte vérifie que le prix de base d'une voiture est strictement positif, car un prix nul ou négatif n'est pas valide.

Car_Option

4. CK_Car_Option_optionPrice CHECK (optionPrice >= 0)

Cette contrainte garantit que le prix d'une option est au moins égal à zéro. Cela permet d'inclure des options gratuites tout en interdisant des prix négatifs.

Modification

La contrainte s'assurant que *emailUserManages* est un *superUser* n'a pas pu être implémentée car cela est impossible avec un CHECK puisque cela demande d'accéder au champ *superUser* de la table *User*.

Performance

5. CK_Performance_maxSpeed CHECK (maxSpeed > 0 AND maxSpeed <= 600)

Cette contrainte assure que la vitesse maximale est réaliste (au-dessus de 0 km/h) et inférieure ou égale à 600 km/h, une limite raisonnable pour les voitures de série.

6. CK_Performance_zeroToHundredTime CHECK (zeroToHundredTime > 0)

Cette contrainte garantit que le temps d'accélération de 0 à 100 km/h est strictement positif. Une valeur négative ou nulle serait invalide.

Engine

7. CK_Engine_horsePower CHECK (horsePower > 0 AND horsePower < 3000)

Cette contrainte vérifie que la puissance d'un moteur est comprise entre 1 et 2999 chevaux. Elle empêche des valeurs irréalistes ou incohérentes.

8. CK_Engine_price CHECK (price > 0)

Cette contrainte impose que le prix du moteur soit strictement positif. Un moteur gratuit ou avec un prix négatif serait incohérent.

Gas

9. CK_Gas_numberOfCylinders CHECK (numberOfCylinders >= 0 AND numberOfCylinders <= 18)

Cette contrainte limite le nombre de cylindres entre 0 (pour des configurations sans cylindres, comme les moteurs électriques) et 18 (nombre maximal observé dans des moteurs de série).

10. CK_Gas_engineDisplacement CHECK (engineDisplacement > 0)

Cette contrainte vérifie que la cylindrée du moteur est strictement positive, car une valeur négative ou nulle serait illogique.

Electric

11. CK_Electric_maxPower CHECK (maxPower > 0)

Cette contrainte s'assure que la puissance maximale d'un moteur électrique est strictement positive. Une puissance nulle ou négative serait invalide.

12. CK_Electric_batteryDistanceCapacity CHECK (batteryDistanceCapacity > 0)

Cette contrainte impose que la capacité d'autonomie d'une batterie (en kilomètres) soit strictement positive. Cela garantit que le véhicule électrique peut parcourir une certaine distance.

Transmission

13. CK_Transmission_numberOfGears CHECK (numberOfGears > 0 AND numberOfGears <= 10)

Cette contrainte limite le nombre de vitesses à un minimum de 1 et un maximum de 10. Cela correspond aux transmissions modernes.

14. CK_Transmission_price CHECK (price > 0)

Cette contrainte garantit que le prix d'une transmission est strictement positif.

Brake

15. CK_Brake_price CHECK (price > 0)

Cette contrainte impose que le prix d'un système de freinage soit strictement positif, car un prix nul ou négatif serait incohérent.

Review

16. CK_Review_grade CHECK (grade > 0 AND grade <= 5)

Cette contrainte limite la note d'une évaluation entre 1 et 5. Cela correspond souvent aux systèmes de notation.

17. CK_Review_date CHECK (date <= CURRENT_DATE)

Cette contrainte s'assure que la date de l'évaluation est dans le passé ou le présent. Une évaluation ayant une date située dans le futur serait illogique.

La contrainte d'intergité s'assurant que la date de *review* est supérieure ou égale à la *releaseDate* d'une voiture n'a cependant pas pu être implémentée car cela est impossible avec un CHECK, puisque le champ *releaseDate* se trouve sur une autre table, *Car*.

4.2 Types de données

Nous avons décidé de ne pas expliquer la limite du nombre de caractères pour les attributs lorsque celle-ci est évidente ou correspond à une logique intuitive, comme la longueur typique d'un nom, d'un nom de modèle ou le texte d'une modification

Brand

- **image VARCHAR(1030)**: L'URL d'une image peut varier en longueur, mais 1030 caractères permettent de couvrir la plupart des cas pour les URL longues¹.

Image

- **image VARCHAR(1030)**: Similaire à la table *Brand*, destiné à stocker des URL d'images.

AppUser

- **email VARCHAR(320)**: 320 caractères est la taille maximale pour un email (64 avant le @ et 255 après : 64 + 1 + 255).
- **password VARCHAR(80)**: La valeur est susceptible de changer car nous ne savons pas encore quel algorithme de hachage nous allons utiliser et c'est ce dernier qui définira cette valeur.

Engine

- **modelName VARCHAR(80)**: Texte limité pour identifier un moteur.

¹ cf. <https://stackoverflow.com/questions/66835898/what-is-the-maximum-length-of-the-profile-picture-url-inside-the-openid-jwt-pic>

- **position POSITION_ENUM:** Peut prendre les valeurs suivantes : “*front*”, “*middle*”, “*rear*”, “*underfloor*”. Cela correspond aux différents endroits où peut se trouver un moteur.

Transmission

- **type TRANSMISSION_TYPE_ENUM:** Énumération pour limiter aux types “*automatic*” ou “*manual*”.
- **drivetrain DRIVETRAIN_ENUM:** Peut prendre les valeurs suivantes : “*rwd*”, “*fwd*”, “*awd*”, “*4wd*”.

5. Description de l’application réalisée

Ce chapitre couvre l'ensemble des fonctionnalités disponibles, du processus de connexion à la gestion avancée des utilisateurs·ices pour les administrateurs·ices.

5.1 Authentification

Pour accéder aux fonctionnalités de notre site, chaque utilisateur·ice doit disposer d’un compte.

- **Création de compte :** L'utilisateur·ice peut créer un compte en fournissant des informations de base telles qu'un nom d'utilisateur·ice, une adresse e-mail et un mot de passe. Une vérification de la validité des données est effectuée pour garantir la sécurité et éviter les duplications.
- **Connexion :** Une fois inscrit·e, l'utilisateur·ice peut se connecter en renseignant ses identifiants. Le système utilise une méthode sécurisée pour valider les informations.
- **Déconnexion :** Une option de déconnexion est disponible dans l'interface utilisateur pour mettre fin à la session de manière sécurisée.



Se Connecter

Email :

Mot de passe :

Se connecter

Pas encore de compte ? [Créer un compte](#)

5.2 Recherche de voitures

La recherche constitue une fonctionnalité centrale de notre site. Les utilisateurs-ices peuvent rechercher une voiture à l'aide de plusieurs critères :

- **Par nom** : Recherche directe en entrant le nom précis d'un modèle.
- **Par marque** : Les utilisateur-ices peuvent sélectionner une marque dans une liste déroulante ou en cliquant sur une section dédiée. Cela redirige vers une recherche de voitures filtrée par cette marque.
- **Par type de moteur** : L'utilisateur-ice peut choisir entre plusieurs types de moteurs (essence ou électrique).
- **Par puissance** : L'utilisateur-ice peut choisir une puissance minimale afin de filtrer les voitures.

Les résultats sont affichés sous forme de liste, accompagnés d'images miniatures et d'informations succinctes sur chaque modèle.

[Accueil](#) [Inscription](#) [Connexion](#) [Marques](#) [Voitures](#)

Recherche de Voitures

Entrez les critères de recherche pour trouver votre voiture préférée.

Nom de la voiture:

Marque:

Type de moteur :

Essence ▾

Puissance minimale :

Puissance en chevaux ▴ ▾

Rechercher

5.3 Fiche détaillée d'une voiture

Lorsqu'un-e utilisateur-ice clique sur un modèle de voiture, iel accède à une fiche détaillée contenant :

- Les **informations sur le modèle de la voiture** : nombre de sièges, date de sortie, prix, image de la voiture
- Les détails de la **spécification** : freins, ABS, transmission, performance, moteur

Cette page constitue également un point d'interaction pour d'autres fonctionnalités, telles que les reviews et les propositions de modification.

Tesla Model S

Nombre de sièges : 5

Date de sortie : 2016-05-10

Prix : 79999 \$€

Images :



Connectez-vous pour proposer une modification.

Spécifications

Spécification 1

Spécification 2

Détails de la Spécification

Freins : ABS Standard

ABS : Oui

Transmission : automatic

Performance : 250 kmh

Moteurs :

- Electric Motor Model S - 1020 ch

Reviews

Amazing Performance - Note : 5/5

The Model S is incredibly fast and comfortable.

Posté par : user1

Connectez-vous pour écrire une review.

5.4 Gestion des reviews

Les utilisateurs·ices peuvent interagir avec les spécifications en :

- **Consultant les reviews existantes** : Chaque spécification peut recevoir des notes et des commentaires de la part d'autres utilisateurs·ices.
- **Ajoutant une review** : L'utilisateur·ice peut attribuer une note et laisser un commentaire sur une spécification spécifique.

Ajouter une Review

Titre :

Contenu :

Note :

Envoyer

Annuler

5.5 Proposition de modifications

Une fonctionnalité clé permet à chaque utilisateur·ice de proposer des modifications sur les données d'une voiture. Par exemple, si une information est erronée ou nécessite une mise à jour, l'utilisateur·ice peut soumettre une suggestion accompagnée d'un commentaire explicatif.

Ces suggestions sont envoyées aux administrateurs·ices pour validation. L'administrateur·ice doit ensuite, si la suggestion est acceptée, modifier à la main les données liées à la voiture. Ce n'est évidemment pas une manière de faire optimale et si l'on souhaite améliorer notre projet il faudrait implémenter une nouvelle section dans

l'admin afin de pouvoir modifier simplement les données d'une voiture ; soit implémenter un CRUD sur le modèle *Car*.

Proposer une modification

Texte :

Le prix me semble faux. Je crois qu'elle vaut 50'000 euros.

Envoyer

Annuler

Connecté en tant que : admin

Gestion des Modifications

Liste des Modifications

- Voiture :** Model S
Proposition : Add autopilot to Model S
Proposée par : user1@example.com
Statut : En attente

Accepter Rejeter
- Voiture :** X5
Proposition : Enhance safety features on X5
Proposée par : user2@example.com
Statut : En attente

Accepter Rejeter

5.6 Gestion des utilisateurs pour les administrateurs

Les administrateurs disposent également d'un module pour gérer les utilisateurs. Ils peuvent :

- **Visualiser la liste des utilisateurs** inscrits.
- **Gérer les droits** des utilisateurs (par exemple, attribuer ou révoquer les privilèges administratifs).
- **Suspendre ou supprimer des comptes.**

Gestion des Utilisateurs

Liste des Utilisateurs

Email	Nom d'utilisateur	Super Admin	Actions
admin@example.com	admin	Oui	<button>Modifier</button> <button>Supprimer</button>
user1@example.com	user1	Non	<button>Modifier</button> <button>Supprimer</button>
user2@example.com	user2	Non	<button>Modifier</button> <button>Supprimer</button>
reviewer@example.com	reviewer	Non	<button>Modifier</button> <button>Supprimer</button>

Ajouter un utilisateur

Email	
Nom d'utilisateur	
Mot de passe	
☐ Super Admin	
<button>Ajouter</button>	

6. Améliorations futures

Bien que notre projet atteigne les objectifs initiaux fixés, plusieurs axes d'amélioration pourraient être envisagés. Ces améliorations, listées ci-dessous, offrent des perspectives intéressantes pour une éventuelle extension du projet :

Gestion des catégories de voitures

Une table dédiée aux catégories de voitures a été créée dans la base de données, permettant une classification des modèles en fonction de critères tels que voitures sportives, voitures familiales, véhicules utilitaires, etc.

Cependant, cette fonctionnalité n'a pas encore été mise en œuvre dans l'application. Une implémentation future pourrait permettre l'affiliation des utilisateurs·ices à une ou plusieurs catégories selon leurs préférences personnelles ou encore la navigation par catégorie, offrant une expérience utilisateur·ice plus personnalisée.

Gestion des options

Une table *Option* a également été définie, offrant la possibilité d'ajouter des fonctionnalités personnalisables aux voitures, telles que des sièges chauffants ou un toit ouvrant.

Les améliorations futures pourraient inclure l'affichage des options disponibles pour chaque voiture, avec leur impact sur le prix total, une interface utilisateur permettant de sélectionner des options lors de la consultation des modèles.

Admin Django

Le framework Django nous met à disposition un admin² qui nous permet de consulter tous les modèles créés et de pouvoir modifier les valeurs des objets ou d'en ajouter. Actuellement, l'admin Django n'est pas pleinement fonctionnel, car l'attribut *is_staff* n'a pas été intégré au modèle *AppUser* et Django a besoin de cet attribut pour faire fonctionner l'admin. Bien que ce module ne soit pas utilisé dans le cadre du projet actuel, son activation pourrait permettre de faciliter la modification des données d'une

² Consultable ici : <http://127.0.0.1:8000/admin/>

voiture pour un admin à la suite d'une suggestion de modification par un·e utilisateur·ice.

Amélioration de l'interface graphique

Bien que le site soit fonctionnel, l'esthétique de l'interface utilisateur·ice pourrait être améliorée afin de rendre l'application plus attrayante, notamment en améliorant les polices, les couleurs, et les animations interactives.

7. Conclusion

Ce projet de dictionnaire automobile interactif nous a permis d'explorer et de maîtriser divers aspects du développement logiciel, depuis la conception de bases de données jusqu'à la création d'une application web complète et fonctionnelle.

Le résultat final offre une plateforme intuitive et performante, permettant aux utilisateur·ices de rechercher et d'évaluer des voitures en fonction de leurs besoins spécifiques. En combinant les technologies modernes telles que Django Rest Framework, VueJS et PostgreSQL, nous avons pu créer un outil robuste et extensible.

Le projet a également mis en lumière l'importance des bonnes pratiques de conception et des contraintes d'intégrité dans les bases de données pour garantir la fiabilité et la cohérence des informations. De plus, l'intégration de fonctionnalités comme les reviews et les suggestions de modifications favorise une expérience utilisateur·ice participative, tout en permettant de maintenir une base de données à jour et pertinente.

8. Annexes

8.1 Guide d'installation

Afin d'installer notre projet à partir de zéro sur votre machine, veuillez suivre ce guide d'installation :

Cloner le repo Github

Notre repo se trouve à l'adresse suivante :
https://github.com/Axwells/ProjetBDR_Groupe4.

La procédure d'installation de notre projet se trouve également dans le README.

Commencer par cloner le projet :

```
git clone git@github.com:Axwells/ProjetBDR_Groupe4.git
```

Backend Django

1. Créer un environnement conda

```
conda create -n bdr python=3.11 anaconda  
conda activate bdr
```

2. Installer Django

```
python -m pip install Django
```

3. Créer votre base de données PostgreSQL

```
psql -U postgres -d postgres -h localhost -p 5666
```

```
> CREATE DATABASE "projet-bdr";
```

```
CREATE DATABASE
```

```
> CREATE USER admin WITH PASSWORD '<password>';
```

```
CREATE ROLE
```

```
> GRANT ALL PRIVILEGES ON DATABASE "projet-bdr" TO admin;
```

```
GRANT
```

```
> exit
```

4. Installer Django REST Framework et CORS middleware

```
pip install djangorestframework djangorestframework-simplejwt  
pip install django-cors-headers
```

5. Migrer pour créer les tables

```
cd carinspector  
python manage.py migrate
```

6. Changer les settings du projet si besoin

Si besoin, adapter les settings suivants dans le fichier *settings.py* à l'intérieur du dossier *mycarinspector*.

```
DATABASES = {  
    'default': {  
        'ENGINE': 'django.db.backends.postgresql',  
        'NAME': 'projet-bdr',          # Replace with your database name  
        'USER': 'postgres',           # Replace with your database user  
        'PASSWORD': 'trustno1',       # Replace with your password  
        'HOST': 'localhost',          # Default is localhost  
        'PORT': '5666',               # Default PostgreSQL port  
    }  
}
```

7. Démarrer le serveur backend

```
python manage.py runserver
```

Par défaut, le backend sera accessible via : <http://127.0.0.1:8000/>.

Front-end (Vue.js)

1. Installer Node.js et npm

Télécharger et installer la version LTS de Node.js du site officiel : <https://nodejs.org/en/download>.

Vérifier que Node.js et npm sont installé correctement :

```
node -v  
npm -v
```

2. Installer les dépendances et Axios

```
cd vue-project  
npm install  
npm install axios
```

3. Démarrer le serveur

```
npm run dev
```

Par défaut, le projet sera accessible via : <http://localhost:5173>.